МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

ЗВІТ

ПРО ЛАБОРАТОРНУ РОБОТУ №1

ТЕМА: «ОСНОВИ СТВОРЕННЯ НАЙПРОСТІШОЇ WEBGL-ПРОГРАМИ»

Виконав:
Студент групи ІО-42
Куліков М. М.

Перевірив:
доц. каф. ІПІ
Родіонов П.Ю.

Київ – 2026

ЛАБОРАТОРНА РОБОТА №1

Тема: «Основи створення найпростішої WEBGL-програми».

Мета: отримати практичні навички програмування WebGL-програм, які дозволяють створювати графічні об’єкти та анімації.

Хід роботи:

Лістинг 1 – Програмний код у файлі LR1.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <script src="scripts.js"></script>
    <link rel="stylesheet" href="style.css">
    <title>LR1</title>
    </head>
  <body>
    <div>
        <h3> Завдання 1-2 </h3>
      <canvas id="canvas-triangle" width="650" height="500"></canvas>
    </div>
    <div>
        <h3> Завдання 3 </h3>
      <canvas id="canvas-two-triangles" width="650" height="500"></canvas>
    </div>
    <div>
        <h3> Завдання 4 </h3>
      <canvas id="canvas-triangle-fan" width="650" height="500"></canvas>
    </div>
  </body>
</html>
```

Лістинг 2 – Програмний код у файлі scripts.js

```
window.addEventListener("load", setup_canvas_triangle);
window.addEventListener("load", setup_canvas_two_triangles);
window.addEventListener("load", setup_canvas_triangle_fan);

function setup_canvas_triangle() {
  const canvas = document.querySelector("#canvas-triangle");
```

```
  const gl = canvas.getContext("webgl");
  if (!gl) {
    alert("Unable to initialize WebGL. Your browser or machine may not support
it.");
    return;
  }

  gl.viewport(0, 0, canvas.width, canvas.height);
  gl.clearColor(0.0, 0.0, 0.0, 0.0);
  gl.clear(gl.COLOR_BUFFER_BIT);
  const vsSource = `
    attribute vec4 aVertexPosition;
    attribute vec4 aVertexColor;
    varying lowp vec4 vColor;
    void main(void) {
      gl_Position = aVertexPosition;
      vColor = aVertexColor;
    }
  `;

  const fsSource = `
    varying lowp vec4 vColor;
    void main(void) {
      gl_FragColor = vColor;
    }
  `;

  const shaderProgram = initShaderProgram(gl, vsSource, fsSource);
  const programInfo = {
    program: shaderProgram,
    attribLocations: {
      vertexPosition: gl.getAttribLocation(shaderProgram, "aVertexPosition"),
      vertexColor: gl.getAttribLocation(shaderProgram, "aVertexColor"),
    },
  };

  const positions = [
     0.0,  0.6,
    -0.6, -0.6,
     0.6, -0.6,
  ];
  const positionBuffer = gl.createBuffer();
  gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
```

```
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(positions), gl.STATIC_DRAW);
  const colors = [
    1.0, 0.0, 0.0, 1.0, // RED
    0.0, 1.0, 0.0, 1.0, // GREEN
    0.0, 0.0, 1.0, 1.0, // BLUE
  ];
  const colorBuffer = gl.createBuffer();
  gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(colors), gl.STATIC_DRAW);
  gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
  gl.vertexAttribPointer(programInfo.attribLocations.vertexPosition,          2,
gl.FLOAT, false, 0, 0);
  gl.enableVertexAttribArray(programInfo.attribLocations.vertexPosition);
  gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
  gl.vertexAttribPointer(programInfo.attribLocations.vertexColor,  4,  gl.FLOAT,
false, 0, 0);
  gl.enableVertexAttribArray(programInfo.attribLocations.vertexColor);
  gl.useProgram(programInfo.program);
  gl.drawArrays(gl.TRIANGLES, 0, 3);
}

function setup_canvas_two_triangles() {
  const canvas = document.querySelector("#canvas-two-triangles");
  const gl = canvas.getContext("webgl");
  if (!gl) {
    alert("Unable to initialize WebGL. Your browser or machine may not support
it.");
    return;
  }

  gl.viewport(0, 0, canvas.width, canvas.height);
   const vsSource = `
    attribute vec4 aVertexPosition;
    attribute vec4 aVertexColor;
    uniform float uAngle;
    varying lowp vec4 vColor;
    void main(void) {
      float x = aVertexPosition.x * cos(uAngle) - aVertexPosition.y * sin(uAngle);
      float y = aVertexPosition.x * sin(uAngle) + aVertexPosition.y * cos(uAngle);
      gl_Position = vec4(x, y, 0.0, 1.0);
      vColor = aVertexColor;
    }
  `;
```

```
const fsSource = `
  varying lowp vec4 vColor;
  void main(void) {
    gl_FragColor = vColor;
  }
`;

const shaderProgram = initShaderProgram(gl, vsSource, fsSource);
 const programInfo = {
  program: shaderProgram,
  attribLocations: {
    vertexPosition: gl.getAttribLocation(shaderProgram, "aVertexPosition"),
    vertexColor: gl.getAttribLocation(shaderProgram, "aVertexColor"),
  },
  uniformLocations: {
    uAngle: gl.getUniformLocation(shaderProgram, "uAngle"),
  },
};

const positions = [
  -0.4,  0.4,
   0.4,  0.4,
  -0.4, -0.4,

  -0.4, -0.4,
   0.4,  0.4,
   0.4, -0.4,
];
const positionBuffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(positions), gl.STATIC_DRAW);

const colors = [
  1.0, 0.0, 0.0, 1.0, // Red
  0.0, 1.0, 0.0, 1.0, // Green
  0.0, 0.0, 1.0, 1.0, // Blue
  0.0, 0.0, 1.0, 1.0, // Blue
  0.0, 1.0, 0.0, 1.0, // Green
  1.0, 1.0, 0.0, 1.0, // Yellow
];
const colorBuffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
```

```
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(colors), gl.STATIC_DRAW);
  let angle = 0.0;
  function render(now) {
    angle += 0.02;
    gl.clearColor(0.0, 0.0, 0.0, 0.0);
    gl.clear(gl.COLOR_BUFFER_BIT);
    gl.useProgram(programInfo.program);
    gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
    gl.vertexAttribPointer(programInfo.attribLocations.vertexPosition,        2,
gl.FLOAT, false, 0, 0);
    gl.enableVertexAttribArray(programInfo.attribLocations.vertexPosition);
    gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
    gl.vertexAttribPointer(programInfo.attribLocations.vertexColor, 4, gl.FLOAT,
false, 0, 0);
    gl.enableVertexAttribArray(programInfo.attribLocations.vertexColor);
    gl.uniform1f(programInfo.uniformLocations.uAngle, angle);
    gl.drawArrays(gl.TRIANGLES, 0, 6);
    requestAnimationFrame(render);
  }
  requestAnimationFrame(render);
}

function setup_canvas_triangle_fan() {
  const canvas = document.querySelector("#canvas-triangle-fan");
  const gl = canvas.getContext("webgl");
  if (!gl) {
    alert("Unable to initialize WebGL. Your browser or machine may not support
it.");
    return;
  }

  gl.viewport(0, 0, canvas.width, canvas.height);
  const vsSource = `
    attribute vec2 aVertexPosition;
    attribute vec4 aVertexColor;
    uniform float uOffsetY;
    varying lowp vec4 vColor;
    void main(void) {
    vec2 postion = aVertexPosition + vec2(0.0, uOffsetY);
      gl_Position = vec4(postion, 0.0, 1.0);
      vColor = aVertexColor;
    }
  `;
```

```
const fsSource = `
  varying lowp vec4 vColor;
  void main(void) {
    gl_FragColor = vColor;
  }
`;

const shaderProgram = initShaderProgram(gl, vsSource, fsSource);
const programInfo = {
  program: shaderProgram,
  attribLocations: {
    vertexPosition: gl.getAttribLocation(shaderProgram, "aVertexPosition"),
    vertexColor: gl.getAttribLocation(shaderProgram, "aVertexColor"),
  },
  uniformLocations: {
    uOffsetY: gl.getUniformLocation(shaderProgram, "uOffsetY"),
  },
};

 const positions = [
  0.0,  0.0,
  -0.3, 0.3,
  0.0, 0.5,
  0.3, 0.3,
  0.4, 0.0,
  0.2, -0.3,
  -0.2, -0.4,
  -0.4, 0.0,
  -0.3, 0.3,
];
const positionBuffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(positions), gl.STATIC_DRAW);

const colors = [
  1.0, 1.0, 1.0, 1.0, // Білий центр
  1.0, 0.0, 0.0, 1.0, // Червоний
  0.0, 1.0, 0.0, 1.0, // Зелений
  0.0, 0.0, 1.0, 1.0, // Синій
  1.0, 1.0, 0.0, 1.0, // Жовтий
  1.0, 0.0, 1.0, 1.0, // Фіолетовий
  0.0, 1.0, 1.0, 1.0, // Блакитний
```

```
    0.5, 0.5, 0.5, 1.0, // Сірий
    1.0, 0.0, 0.0, 1.0, // Червоний (для замикання)
  ];
  const colorBuffer = gl.createBuffer();
  gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(colors), gl.STATIC_DRAW);
  let time = 0.0;

  function render(now) {
    time += 0.01;
    const offsetY = Math.sin(time) * 0.5;
    gl.clearColor(0.0, 0.0, 0.0, 0.0);
    gl.clear(gl.COLOR_BUFFER_BIT);
    gl.useProgram(programInfo.program);
    gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
    gl.vertexAttribPointer(programInfo.attribLocations.vertexPosition,        2,
gl.FLOAT, false, 0, 0);
    gl.enableVertexAttribArray(programInfo.attribLocations.vertexPosition);
    gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
    gl.vertexAttribPointer(programInfo.attribLocations.vertexColor, 4, gl.FLOAT,
false, 0, 0);
    gl.enableVertexAttribArray(programInfo.attribLocations.vertexColor);
    gl.uniform1f(programInfo.uniformLocations.uOffsetY, offsetY);
    gl.drawArrays(gl.TRIANGLE_FAN, 0, 9);
    requestAnimationFrame(render);
  }
  requestAnimationFrame(render);
}

function initShaderProgram(gl, vsSource, fsSource) {
  const vertexShader = loadShader(gl, gl.VERTEX_SHADER, vsSource);
  const fragmentShader = loadShader(gl, gl.FRAGMENT_SHADER, fsSource);
  const shaderProgram = gl.createProgram();
  gl.attachShader(shaderProgram, vertexShader);
  gl.attachShader(shaderProgram, fragmentShader);
  gl.linkProgram(shaderProgram);
  if (!gl.getProgramParameter(shaderProgram, gl.LINK_STATUS)) {
    alert("Unable    to    initialize    the    shader    program:    "    +
gl.getProgramInfoLog(shaderProgram));
    return null;
  }
  return shaderProgram;
}
```

```
function loadShader(gl, type, source) {
  const shader = gl.createShader(type);
  gl.shaderSource(shader, source);
  gl.compileShader(shader);
  if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
    alert("An error occurred compiling the shaders: " +
gl.getShaderInfoLog(shader));
    gl.deleteShader(shader);
    return null;
  }
  return shader;
}
```

Лістинг 3 – Програмний код у файлі style.css

```
body {
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            min-height: 100vh;
            margin: 0;
            padding: 20px 0;
        }

div {
    margin-bottom: 30px;
    text-align: center;
    padding: 10px;
    border: 1px solid #222;
    border-radius: 10px;
}

canvas {  border: 2px solid #fff; }
```

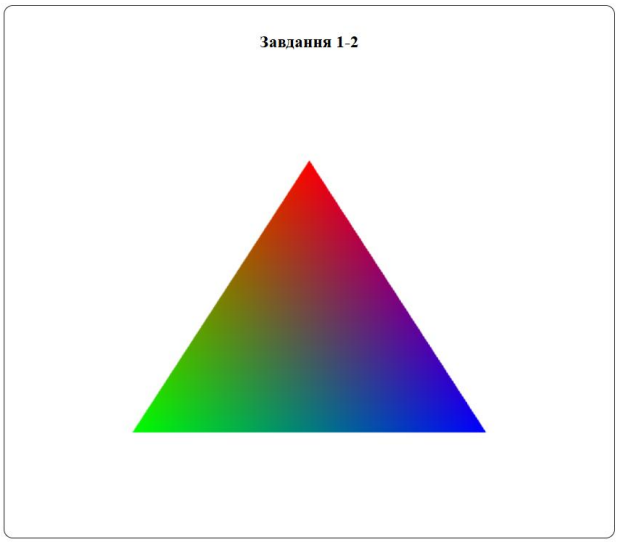


Рисунок 1 – Створений кольоровий трикутник

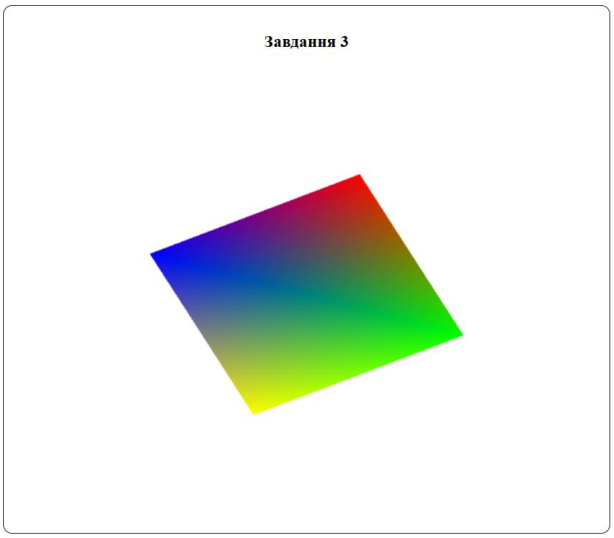


Рисунок 2 – Створений прямокутник

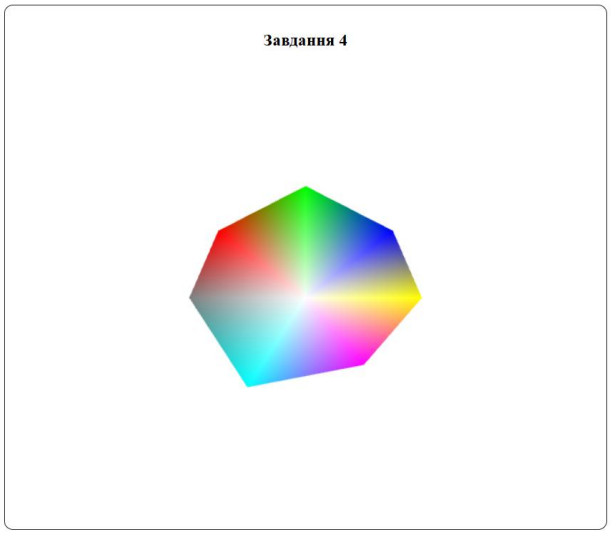


Рисунок 3 – Створена довільна графічна фігура

ВИСНОВКИ:

Отже, під час виконання лабораторної роботи було отримано навички створення найпростіших WebGL програм, розроблено HTML сторінку, де відбувається рендеринг кольорового трикутника, прямокутника, що крутиться, а також довільної фігури, яка рухається вниз та угору.

Отримані результати підтверджують коректність розуміння основних етапів графічного конвеєра WebGL та практичних прийомів для створення й анімації графічних примітивів у веб-браузері без використання додаткових плагінів.

Проблем під час виконання лабораторної роботи не виникло.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ:

Використані веб-ресурси:

1. MDN Web Docs. WebGL API. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API
2. WebGL guide (part 1/2). URL: https://xem.github.io/articles/webgl-guide.html
3. WebGL2 Fundamentals. URL: https://webgl2fundamentals.org/